Running the server as a container

The published image is `registry.badassium.com/fatalexception/yarg-song-server`, tagged `latest` and the commit short sha. It is a distroless static binary — **8.6 MB**, no shell, running as `nonroot`.

This document records the first real deployment, on vault2, 2026-09-05.

What this deployment proved, that nothing had proved before

The multi-arch CI job verifies the image config's architecture and the ELF machine type of the binary inside. Both are checks on *bytes*. Until this deployment **the published image had never actually been run** — not by CI, not by anyone. It now has:

```
library indexed songs=22 distinct_charts=22 duplicate_packages=0 problems=0 took=7ms
listening addr=:8080 songs=/songs data=/data version=827e0fe
```

`version=827e0fe` is how you know the image is the commit it claims to be.

It also moved the server off the development machine for the first time. Every earlier measurement was `127.0.0.1`, which cannot fail in any of the ways a real network can.

The deployment

```
# on the host
mkdir -p /mnt/cache/appdata/yarg-song-server/songs \
         /mnt/cache/appdata/yarg-song-server/data
chown -R 65532:65532 /mnt/cache/appdata/yarg-song-server/data

docker run -d --name yarg-song-server \
  --restart unless-stopped \
  -p 8099:8080 \
  -v /mnt/cache/appdata/yarg-song-server/songs:/songs:ro \
  -v /mnt/cache/appdata/yarg-song-server/data:/data \
  registry.badassium.com/fatalexception/yarg-song-server:latest
```

Three things there are not decoration:

- **`chown 65532`.** The image is distroless and runs as `nonroot`, uid 65532. `/data` holds the on-demand pack cache and must be writable by that uid or the server starts fine and then fails the first time somebody asks for a song that is stored as anything but a `.sng` — a loose folder, a `.zip` or a `.7z`, all of which are packed on demand. `/songs` stays root-owned and is mounted `:ro`, because the server never writes to the library and should not be able to.
- **`/mnt/cache/...`, not `/mnt/user/...`.** An absolute pool path, not the shfs union. This host has a documented history of SQLite-on-FUSE corruption, and `/mnt/user` was **99% full** at deployment (568 GB of 29 TB) while the cache pool had 1.4 TB free.
- **`-p 8099:8080`.** 8099 was free; the container listens on 8080 internally. Checked against `ss -ltn` rather than assumed — this host had 59 containers running and 76 ports already in LISTEN.

Registry authentication

The host's stored credential was stale and `docker pull` failed with `unauthorized: HTTP Basic: Access denied`. A GitLab personal access token with the `api` scope works for a registry pull — measured; `read_registry` is the documented scope but was not required here.

Log in without putting the secret in `argv`, where `ps` would show it:

```
docker login registry.badassium.com -u <user> --password-stdin <<'ENDTOK'
<token>
ENDTOK
```

The token comes from 1Password at run time and is never written to a file, a commit, or this document.

End-to-end result, 2026-09-05

Step	Result
Pull and start on vault2	running, 0 restarts, indexed 22 songs in 7 ms
`GET /healthz` from the host	200
`yarg-sync` from ENG-1 over Tailscale	22 downloaded, 0 failed, 229,515 bytes in **341 ms**
Byte count vs the localhost run	**identical** — same archives, different machine
Second sync	0 downloaded, 22 already present, **0 bytes**, 19 ms
Unmodified YARG on the synced folder	**20 accepted, 2 refused**

The two refusals were the same two, for the same reasons, as every earlier run — `no notes found` and `no audio accompanying the chart file`, both independently flagged by our own scanner.

The client reached the server by its **Tailscale name**, not a LAN literal, so the same command works from anywhere ENG-1 happens to be. A `192.168.2.x` address would have worked at the time of the test and failed from the office.

The pack cache bound, verified here rather than in a test

`pack_cache_max` shipped with six red-proofed tests and did not work. Every test inserted, so every test reached eviction through the insert path; this deployment's cache was already full, every request was a hit, nothing was ever packed, and the bound was never enforced. Deploying it found that in under a minute.

After the fix (`6bcf4ec` , enforce at start as well as on insert), running deliberately tight at **102,400 bytes** against a library that packs to 229,515:

Moment	Cache
Before restart, from the broken build	229,515 bytes, 22 archives
Immediately after start, having served nothing	**67,681 bytes, 6 archives**
After serving all 22 songs	67,681 bytes, 6 archives
Songs the client received	**22 of 22, 0 failed**

Three things in that table, in order of importance:

- The cache dropped from 22 archives to 6 **before a single request arrived**. That is the start-up enforcement, and it is the case the tests all missed.

- It stayed at 6 while serving 22 songs, so eviction kept pace with insertion rather than merely happening once.
- **Every song was still delivered and verified.** Sixteen of the twenty-two had to be re-packed mid-sync because eviction had removed them, and the client — which re-derives each song's identity from the bytes it receives — accepted all of them. That is the concrete evidence that eviction costs a re-pack and never data.

> **The conclusion drawn from that third bullet was too strong, and the number in it was a clue nobody read.** "Eviction costs a re-pack and never data" is true; "and therefore the re-packed archive is the same archive" does not follow, and was false. Those same sixteen songs came back with *different bytes* — the packer drew a random mask per call — which the client could not notice, because it verifies the chart's identity and the chart is copied byte for byte. Identity survived; the archive did not. The next day the same sixteen turned up as sixteen SHA-256 mismatches between two machines, and that is what named the defect. Fixed on 2026-09-06 by deriving the mask; `docs/TEST-CORPUS.md` has the run. **A client that verifies identity will accept a byte stream that a cache, an `ETag` and a `Range` resume all require to be stable — so client acceptance is not evidence of stability.**

What this still does not prove

- ~~**arm64 has never been executed.**~~ **Closed 2026-09-06.** Both the cross-compiled arm64 binary and the published arm64 container image ran on a Raspberry Pi 4 Model B Rev 1.5 (`aarch64` , Debian 13 trixie), each indexing the 22-song corpus in about 25 ms, and the archives they served were byte-for-byte identical to every x86-64 sync. See `docs/TEST-CORPUS.md` , "The ARM leg". Note the caveats there: it was a Pi 4 rather than the 3B+ (that board is dead), the machine was one-shot and wiped, and the image was transported by `docker save` / `load` rather than pulled directly on the Pi.
- ~~**The Phase 2b exit criterion needs two clients.**~~ **Closed 2026-09-06.** YARG was installed on r7-desktop and both machines were scanned: 20 accepted, 2 refused on each, the same two songs. Doing it is what found the determinism defect above — see `docs/TEST-CORPUS.md` , fourth oracle run.
- ~~**One library, 22 songs, 224 KB.**~~ **Partly closed 2026-09-06.** Index time is linear at 0.57-0.59 ms/song and memory fits `13 MB + 6.5 KB/song` , measured at 1,000 / 10,000 / 31,109 songs; packing streams, with resident memory flat at 16.5 MB while serving 1.17 GB, and the cache bound held at 11x oversubscription. See `docs/SCALE.md` . **Still open:** every song in that experiment is synthetic, so a real library's *variety* is untested, and the oracle has never run at scale - which is the measurement most likely to find a rejection category we do not flag.

Re-deployed 2026-09-06 on `c623dae`, and what it proved beyond the fix

Every number above was taken from a server that packed with a random mask (`6bcf4ec` was running; an earlier revision of this document said `827e0fe`, which was the deployment before it). Pipeline 2259 published `c623dae` and the host was re-pulled:

```
docker pull registry.badassium.com/fatalexception/yarg-song-server:latest
docker rm -f yarg-song-server
rm -rf /mnt/cache/appdata/yarg-song-server/data/packs   # start with an empty cache on
purpose
docker run -d ...                                       # identical arguments to the first
deploy
```

The stored registry credential from 2026-09-05 was still valid, so no fresh login was needed. Wiping the pack cache first is deliberate: every archive is then packed by the new binary, and nothing served afterwards is a leftover of the random-mask era.

```
library indexed songs=22 distinct_charts=22 duplicate_packages=0 problems=0 took=11ms
pack cache bounded max_bytes=2147483648
listening addr=:8080 songs=/songs data=/data version=c623dae
```

Then five independent syncs were compared file by file:

Client	Server	Result
ENG-1	ENG-1, built from the working tree (windows/amd64)	reference
ENG-1	same, after wiping the whole pack cache	identical
r7-desktop	ENG-1's server	identical
ENG-1	**the vault2 container (linux/amd64)**	**identical**
r7-desktop	**the vault2 container (linux/amd64)**	**identical**

110 archives, two client machines, two servers built for different operating systems and by different toolchains, and one deliberate cache wipe — all one set of bytes. **The derivation is not platform-dependent**, which the fix needed to be but which nothing had shown until this run. Unmodified YARG on the container's output: **20 accepted, 2 refused**, the same two songs.

Worth stating plainly because it is the standard this project holds: the last row of that table is an identity established by hashing every file, not by arguing that identical inputs must give identical results.

Re-deployed 2026-09-06 on `423902b`, with archive ingest

The library on the host was replaced with the 23-case corpus, which includes `23-zipped.zip` — a song delivered as an archive rather than a folder — and the pack cache was wiped so nothing served afterwards was a leftover of the previous image.

```
library indexed songs=23 distinct_charts=23 duplicate_packages=0 problems=0 took=9ms
listening addr=:8080 songs=/songs data=/data version=423902b
```

Check	Result
`/version`	`423902b`
`/healthz`	200
The archive-sourced song in the catalog	`name="Zipped"`, `source_path="23-zipped.zip"`, **0 issues**
`yarg-sync` from an empty folder	23 downloaded, 0 failed, 238,215 bytes in 399 ms
Every archive vs the locally built server	**23 / 23 identical**

That last row is the container-independence property crossing a build platform: the song the **linux/amd64 container** ingested from inside a `.zip` is byte-identical to the one a **windows/amd64** build produced from the loose folder. Nothing about the container it arrived in, or the machine that packed it, reaches the bytes a client receives.

The host's stored registry credential was still valid, so no fresh login was needed.

Re-deployed 2026-09-06 on `077f36e`, and the hostile-archive fix measured on the real thing

Pipeline 2278 published `077f36e` (archive-ingest hardening). Two things were worth measuring here that a test cannot reach: whether a code change altered the bytes a client receives, and whether the new "this archive is unreadable" report actually arrives where an operator would see it.

`latest` **was confirmed to be the commit before it was run**, not assumed:

```
docker pull …:latest
docker pull …:077f36eb        # note EIGHT characters - CI tags with CI_COMMIT_SHORT_SHA
docker image inspect …:077f36eb --format '{{.Id}}'   # same image id as :latest
```

That the tag is eight hex characters and the git short sha is seven is a small trap: a `:077f36e` pull fails with `manifest unknown`, which reads like a missing image rather than a mistyped tag. `GET /api/v4/projects/53/registry/repositories/14/tags` lists what exists.

Nothing about a code change reached the bytes

The pack cache was hashed **before** the upgrade, wiped, and hashed again after the new binary had re-packed all 23 songs:

```
sha256sum *.sng | sort -k2 > /tmp/packs-423902b.txt   # before
# … pull, rm -f, rm -rf data/packs, docker run …
sha256sum *.sng | sort -k2 > /tmp/packs-077f36e.txt   # after
diff /tmp/packs-423902b.txt /tmp/packs-077f36e.txt    # no output
```

Check	Result
`/version`	`077f36e`
`/healthz`	200
Index	23 songs, 23 distinct charts, **0 problems**, 9 ms
23 archives re-packed on a wiped cache, vs the `423902b` cache	**byte-for-byte identical**
`yarg-sync` from ENG-1 over Tailscale	23 downloaded, 0 failed, 238,215 bytes in 312 ms
The 23 files the client received, hashed, vs the 23 packs on the server	**identical set**

The determinism property now holds **across a code change**, which is a different claim from the ones already recorded — those compared machines, operating systems and architectures at a fixed commit. Byte-stability across upgrades is what a client's `ETag` and `Range` resume actually depend on in practice, since a server gets upgraded far more often than it changes CPU.

The silently-ignored song, reproduced and then observed being reported

`077f36e` exists because a zip written with **backslash** separators used to disappear from a library with no message. A probe archive was built to be exactly that shape — `song.ini`, `notes.mid` and `song.ogg` under `Backslash Song\`, written with the separator stored verbatim — dropped into the host's library, and the server restarted.

Building the probe is itself instructive. Python's `zipfile` rewrites `os.sep` to `/` in `ZipInfo.__init__`, so `ZipInfo("Backslash Song\\song.ini")` silently produces a *forward* slash and the probe proves nothing; the name has to be assigned after construction. And `namelist()` normalises backslashes when reading back, so the file looks wrong even when it is right — the only honest check is to count `0x5C` bytes in the file itself. **Both the writer and the reader hide the thing being tested**, which is the same shape as the defect: a convenient view of an archive is not the archive.

```
level=INFO  msg="library indexed" songs=23 distinct_charts=23 duplicate_packages=0
problems=1
level=WARN  msg="could not index" path=24-backslash.zip
            err="scan: archive contains a song.ini or a chart but no song folder could be
read
                 from it; if it was made by an old Windows tool it may use backslash path
                 separators - re-zipping it will fix that"
```

It appears in `/api/v1/library` under `problems` as well as in the log, so an operator with no shell on the host still sees it. Before this commit the same file produced `problems=0` and no mention anywhere.

The probe was removed afterwards and the library restored to the 23-case corpus — a permanent `problems=1` would make every future index report ambiguous. `0 problems`, `restarts=0`, health 200 at the end.

Re-deployed 2026-09-07 on `64b42af`, and the first test with real concurrent clients

`64b42af` is the build that stopped the server 404-ing songs that exist (see ADR-002, "Concurrency"). Deployed the same way: confirm the tag is the commit by image id, wipe the pack cache, recreate.

Take the eight-character tag from `git rev-parse`, do not type it from memory. The image tag is `CI_COMMIT_SHORT_SHA` — eight characters — and the git short sha is seven. Guessing the eighth produced `64b42af7` and a `manifest unknown`, which reads like a missing image rather than a typo. The sha is `64b42af1b368…`, so the tag is `64b42af1`, and its image id matched `:latest` exactly.

Check	Result
`/version`	`64b42af`
`/healthz`	200, `restarts=0`
Index	23 songs, 23 distinct charts, **0 problems**, 15 ms
23 archives re-packed on a wiped cache, vs the `077f36e` cache	**byte-for-byte identical**

That is determinism holding across a **second** code change, on top of the machines, operating systems and architectures already covered.

Four real clients at once, over the network

Every concurrency measurement until now used goroutines against an in-process test server. Four `yarg-sync` processes were run simultaneously from ENG-1 against the vault2 container over Tailscale:

Client	Result
1	23 downloaded, 0 failed, 238,215 bytes, 456 ms
2	23 downloaded, 0 failed, 238,215 bytes, 448 ms
3	23 downloaded, 0 failed, 238,215 bytes, 483 ms
4	23 downloaded, 0 failed, 238,215 bytes, 449 ms

Every file byte-identical across all four clients, compared by SHA-256 per filename rather than by count or total. Four simultaneous clients cost about 40 ms each against the ~410 ms a single client took earlier, so nothing here is serialising badly.

What this run does NOT test, stated because it would be easy to read it as more than it is: this deployment's cache bound is 2 GB against a 238 KB library, so **nothing is ever evicted** and the defect `64b42af` fixes could not fire here even on the old build. This exercises the concurrent *serve* path over a real network; the eviction race is covered by `internal/httpapi/concurrency_test.go`, which has to bound the cache to a single archive to provoke it at all. A Pi with a small SD card and a large library is where the two conditions meet, and that combination is still unmeasured on real hardware.
